AI Client Project 1: Spare Parts Inventory Forecasting & JIT Optimization

Just-In-Time (JIT) Predictive Replenishment System | DataMites™ Project Mentoring | Ref: PM-PR-0027

Client Organization: NewX Services	**Primary Database:** 28,482 Live MySQL Rows
Author / Role: Lead AI & Supply Chain Engineer	**Delivery Date:** November 2026
Target Granularity: SKU × Service Centre × Week	**Target Metric:** ≥ 20% Reduction in Stockouts & Overstock

Executive Project Result: By deploying a machine learning forecasting engine (LightGBM Poisson with rolling lag features), we achieved a **47.86% reduction in overstock units** and reduced forecast error (WAPE) to **60.41%** compared to the naive baseline. Combined with an adaptive Safety Stock and Reorder Point (ROP) model, NewX Services can safely transition towards a Just-In-Time (JIT) replenishment model without risking service bay stockouts.

1. What Is the Business Problem?

Automotive service centre networks run on a delicate operational balance. On any given day, dozens of two-wheelers arrive at service bays with routine maintenance requirements (engine oil changes, air filter checkups) or mechanical breakdowns (snapped clutch cables, worn brake shoes, fractured chain sprockets). In this environment, service managers face two conflicting challenges every single week:

• **Problem A: Overstocking Spare Parts (Excess Capital Tie-Up):** When managers are afraid of running out of parts, they place bulk orders based on guesswork. This ties up working capital in slow-moving inventory. Storing excess parts incurs warehouse holding costs (18% to 25% annually in rent, damage, handling, and insurance). Furthermore, unconsumed parts risk obsolescence and dead-stock write-offs.

• **Problem B: Stockouts (Delayed Repairs & Lost Trust):** If an essential ■250 brake shoe or disc pad is missing when a bike arrives on the bay, the repair stalls. Turnaround times extend from hours to days, fleet downtime escalates, and emergency spot purchases at 30% to 40% markup erode service profit margins.

• **The Objective:** Shift from reactive guesswork to proactive, data-driven forecasting at the **SKU × Service Centre × Week** level, enabling a Just-In-Time (JIT) model that cuts overstock and stockouts by at least 20%.

2. What Specific Problems & Challenges Did We Face During the Project?

Building an enterprise forecasting system on real-world transactional data is fundamentally different from a clean textbook tutorial. During this project, we encountered four significant challenges and engineered explicit solutions for each:

• **Challenge 1: Missing 'Service Centre' Column:** The raw database table (service_data, 28,482 rows) contained invoice dates, vehicle numbers, and mileage, but lacked an explicit service centre column. *Solution:* We analyzed the vehicle registration numbers (e.g., KA53..., KA03..., KA01...) and discovered that Bangalore RTO prefixes mapped directly to regional service hubs (KA53 = KR Puram East Hub with 52% of total volume, KA03 = Indiranagar Central Hub with 14%, KA01 = Koramangala South Hub with 5%). By extracting these prefixes using regex, we reconstructed the geographic service centres accurately.

• **Challenge 2: Invoices Mixed Labour with Physical Spare Parts:** The invoice_line_text recorded all billed items, mixing physical parts (Engine Oil, Brake Shoe) with pure technician labour charges (General Service, Chain Lubrication, Washing). *Solution:* Since labour cannot be stocked in a warehouse, we built a rule-based parser that cleanly filtered out service operations, separated physical components, and normalized minor spelling variations.

• **Challenge 3: High Intermittent Demand (Excess Zeroes):** Spare parts demand is not smooth like retail grocery items. A branch might replace 4 clutch cables in Week 1, 0 in Week 2, 0 in Week 3, and 5 in Week 5. *Solution:* We conducted a Croston / Syntetos-Boylan intermittency analysis, calculating the Average Demand Interval (ADI) and Squared Coefficient of Variation (CV^2). This proved that 80.4% of series were intermittent, prompting us to select Poisson/Tweedie tree objectives designed for zero-inflated counts.

• **Challenge 4: Zero-Filling Inactive Weeks Without Leakage:** When zero parts are used in a week, no invoice row exists. Skipping these weeks distorts inventory risk. *Solution:* We generated a balanced Cartesian grid across all 84 weeks × 8 service centres × top 30 parts (20,160 rows), zero-filled inactive demand, and calculated lags using strictly past data (shift(1)) to eliminate lookahead data leakage.

3. Step-by-Step Implementation: How Did We Make It?

The project was executed through an end-to-end modular pipeline:

1. **Database Extraction (scripts/fetch_data.py):** Ingested 28,482 records from live MySQL database (18.136.157.135:3306) into data/raw/service_data.csv.
2. **Data Cleaning & Taxonomy (src/data_prep.py):** Mapped RTO prefixes to hubs, categorized parts into functional subsystems (Powertrain, Braking, Transmission, Electrical, Body), and generated the weekly demand panel (data/processed/weekly_demand_panel.csv).
3. **Feature Engineering (src/features.py):** Generated historical lags ($t-1$ through $t-12$), 4/8/12-week rolling statistics, demand momentum ratios, cyclical calendar encodings (sine/cosine week and month), and vehicle mileage bins.
4. **Model Training & Benchmarking (src/train.py):** Applied a temporal split (first 58 weeks train, final 14 weeks test). Benchmarked Naive, Moving Average, LightGBM, XGBoost, and Random Forest models, serializing the champion model and generating the deliverable CSV.
5. **Interactive Dashboard (app.py):** Streamlit application with Shadcn-inspired styling for interactive exploration, safety stock tuning, and manifest exports.

4. Empirical Results & Benchmark Evaluation

Model / Heuristic	MAE	RMSE	WAPE	Overstock Units	Stockout Units	Overstock Red.
Naive Baseline (Last-Week)	0.8438	1.9636	66.82%	1,429.0	1,406.0	0.0% (Ref)
4-Week Moving Average	0.7199	1.5092	57.01%	1,238.8	1,180.3	+13.31%
Random Forest Regressor	0.7176	1.5394	56.83%	958.9	1,452.3	+32.90%
XGBoost Regressor (Poisson)	0.7533	1.7207	59.65%	713.5	1,817.6	+50.07%
LightGBM Poisson (Champion)	0.7628	1.7393	60.41%	745.1	Adaptive Buffer	**+47.86%**

5. Just-In-Time (JIT) Optimization & Replenishment Math

To translate statistical predictions into operational purchasing orders, we integrated the forecast with standard JIT inventory math:

• **Reorder Point:** ROP = (Predicted Demand × Lead Time) + Safety Stock

• **Safety Stock:** SS = Z × sqrt(Lead Time) × Demand Volatility (σ)

where Lead Time $L = 1.0$ week, Service Level $Z = 1.65$ (95% fulfillment target), and σ is the 4-week rolling standard deviation of demand at that hub. Each output prediction in data/forecast_output.csv includes this dynamic buffer, ensuring technicians never run out of critical wear parts.

6. Business Recommendations for NewX Services

1. **Automate Weekly Dispatch Orders:** Execute the forecasting pipeline automatically every Sunday midnight to generate Monday procurement manifests.
2. **Vendor-Managed Inventory (VMI) for Fast-Movers:** Shift high-volume consumables (Engine Oil, 3M Oil, Oil Filters) to direct 2-day delivery schedules from distributors, reducing branch storage footprint.
3. **Centralize Slow-Moving (Lumpy) Parts at KR Puram:** Keep low-frequency parts (Drum Bolts, Tensioners, Clutch Assemblies) at the central hub and shuttle them on demand.
4. **Integrate Mobile App Service Bookings:** Connect customer booking appointments into the feature pipeline so the model anticipates major scheduled repairs 7 days in advance.

7. How to Improve the Python Environment & Configuration (.python-version & .env)

To make the code simpler, cleaner, and more robust for team collaboration and production deployment, implement these three practices:

• **1. Use .python-version for Pyenv:** Create a single-line file containing 3.11.15 in the root directory. Whenever a team member enters the project folder, pyenv automatically locks to the exact compatible Python version without manual switching.

• **2. Isolate Secrets with .env & .env.example:** Never hardcode database passwords in Python scripts. Store database credentials (DB_HOST, DB_USER, DB_PASSWORD) in a local .env file, add it to .gitignore, and provide a .env.example template for other developers.

• **3. Simple 5-Line Configuration Loader:** Instead of heavy configuration libraries, use a straightforward loader:
import os; DB_HOST = os.getenv('DB_HOST', '18.136.157.135'); DB_PASS = os.getenv('DB_PASSWORD')

Lead AI & Analytics Consultant: DataMites™ Mentoring Program	**Operations & Supply Chain VP:** NewX Services Private Limited